1. AWS LAMBDA

Q: What is Lambda and how is it different from EC2?
Lambda is a serverless compute service — you upload code, and AWS runs it in response to events without you managing servers. You're billed per invocation + duration (ms-level billing), not for idle time. EC2 is a virtual machine you provision and pay for as long as it's running, regardless of whether it's doing work. Lambda is best for short, event-driven, bursty workloads; EC2 is better for long-running, stateful, or highly custom-environment workloads.

Q: What are Lambda's key limits?
Max timeout 15 minutes, max memory 10,240 MB, deployment package 50MB zipped (250MB unzipped, more via container images up to 10GB), /tmp up to 10GB, default concurrent execution limit 1,000 per account (soft limit, raisable).

Q: What triggers Lambda, sync vs async?

- Sync: API Gateway, ALB, Cognito triggers — caller waits for response.
- Async: S3 events, SNS, EventBridge — Lambda returns immediately, AWS handles retries (2 retries by default) and can route failures to a DLQ or Destination.
- Poll-based: SQS, Kinesis, DynamoDB Streams — Lambda service polls the source and invokes in batches.

Q: In your project — S3 upload → Lambda → Bedrock → DynamoDB. Walk me through it.
S3 event notification fires on ObjectCreated for a specific prefix/suffix → invokes Lambda asynchronously → Lambda reads the object, calls Bedrock's InvokeModel API to process/summarize it → writes result to DynamoDB with the object key as partition key → optionally emits a DynamoDB Stream event for downstream processing. I'd wrap Bedrock calls in retry/backoff logic since throttling is common, and make the DynamoDB write idempotent using a conditional expression on a unique request ID so retries don't create duplicate records.

Q: What is a cold start and how do you reduce it?
A cold start happens when Lambda has to initialize a brand-new execution environment: download code, start the runtime (Firecracker microVM), then run your init code (imports, DB connections) before the handler executes. This adds latency (100ms–several seconds depending on runtime/package size). Mitigations: **Provisioned Concurrency** (pre-warms N environments, costs more but guarantees low latency), **SnapStart** (Java only — snapshots the initialized environment and restores from it), keeping deployment packages small,

avoiding heavy imports at cold-start time, and moving SDK client initialization outside the handler so it's reused across warm invocations.

Q: Reserved vs Provisioned concurrency — what's the difference?
Reserved concurrency caps/guarantees a max number of concurrent executions for a function (protects other functions from being starved, and protects downstream systems like RDS from overload) — but does NOT pre-warm anything, so cold starts still happen. Provisioned concurrency pre-initializes a set number of execution environments so invocations hit warm environments immediately — this is what actually reduces cold-start latency.

Q: How do you handle idempotency if the same S3 event fires Lambda twice?
Use a conditional write in DynamoDB keyed on a unique identifier (e.g., S3 object ETag or a request ID) with ConditionExpression: attribute_not_exists(id) — so a duplicate invocation trying to write the same record fails safely instead of double-processing. For side effects like calling Bedrock, I'd check if a result already exists before making the call.

---

2. AMAZON S3

Q: What is S3 and what's the durability guarantee?
S3 is an object storage service offering 99.999999999% (11 nines) durability by redundantly storing objects across multiple devices in multiple AZs within a region.

Q: Storage classes — when do you use each?
Standard (frequent access), Standard-IA/One Zone-IA (infrequent access, cheaper storage but retrieval fee), Intelligent-Tiering (auto-moves objects between tiers based on access patterns — good when access patterns are unpredictable), Glacier Instant/Flexible/Deep Archive (archival, minutes-to-hours retrieval, very cheap storage).

Q: How does S3 trigger Lambda only for .pdf uploads in a specific folder?
Configure an S3 Event Notification on the bucket with a prefix filter (e.g., uploads/) and suffix filter (.pdf), targeting the Lambda function. S3 grants itself invoke permission on the Lambda via a resource-based policy.

Q: How do you let users upload large files directly and securely from a browser?
Generate a **presigned URL** (via Lambda, using the SDK, with a short expiry and the IAM role's permissions) that the browser uses to PUT directly to S3, bypassing your backend for the actual bytes. For files over ~100MB, use **multipart upload** — split the file into parts, upload in parallel, and S3 assembles them; if a part fails, only that part is retried, not the whole file.

Q: What's S3's consistency model?
Since December 2020, S3 provides **strong read-after-write consistency** for all operations (PUTs and DELETEs) — a GET immediately after a PUT will always return the latest version. Before this, S3 had eventual consistency for overwrite PUTs and DELETEs.

Q: Two Lambdas write to the same S3 key concurrently — what happens?
S3 doesn't lock objects — the last write wins. If versioning is enabled, both writes create separate versions and neither is lost, giving you a way to recover. Without versioning, one write silently overwrites the other, so if concurrent writes to the same key are a real risk, I'd enable versioning or use unique keys per write.

---

3. AWS STEP FUNCTIONS

Q: Why use Step Functions instead of chaining Lambdas manually?
Step Functions gives you visual, stateful orchestration with built-in retry/catch logic, parallel execution, and native integration with 200+ AWS services — without writing glue code. If you chain Lambdas manually, you have to build your own state tracking, retry logic, and error handling, and you lose visibility into where a multi-step process actually failed. Step Functions gives you a full execution history in the console for debugging.

Q: Standard vs Express workflows?
Standard: exactly-once execution, up to 1 year duration, priced per state transition, full execution history retained — good for long-running or auditable workflows. Express: at-least-once execution, max 5 minutes, priced by requests + duration/memory (like Lambda), much cheaper at high volume, minimal history (via CloudWatch Logs) — good for high-volume, short, event-processing workloads (e.g., IoT data, streaming).

Q: How would you process 10,000 items with a concurrency cap?
Use a **Map state** with MaxConcurrency set to your desired cap (e.g., 50) — Step Functions will iterate the array and run the sub-workflow for each item without exceeding that concurrency, which is critical if a downstream service like Bedrock or DynamoDB has throughput limits.

Q: How do you handle a step that fails intermittently, like Bedrock throttling?
Use a Retry block on that state specifying the error type (e.g., ThrottlingException), an interval, a backoff rate, and max attempts. If it still fails after retries, a Catch block routes to a fallback state (e.g., notify via SNS, or write a "failed" status to DynamoDB) instead of failing the whole execution ungracefully.

Q: Standard vs Express — exactly-once vs at-least-once, why does it matter?
Standard workflows use a durable, checkpointed execution model so each state transition is only ever processed once — ideal when duplicate side effects (like double-charging or double-writing) are unacceptable. Express trades that guarantee for much higher throughput and lower cost, so it's only safe when your downstream steps are idempotent.

---

4. AMAZON DYNAMODB

Q: DynamoDB vs RDS — when do you pick which?
DynamoDB is NoSQL, schema-less (per item), and scales horizontally with predictable low-latency performance at any scale, but you must design your table around known access patterns upfront. RDS is relational, supports complex joins/transactions/ad-hoc queries, but scaling and latency get harder at very high throughput. I chose DynamoDB because my access patterns were simple key-based lookups (fetch by request ID / user ID) and I needed single-digit millisecond latency without managing scaling.

Q: Partition key vs sort key?
Partition key determines which physical partition an item is stored on (via hashing) — it drives distribution and scalability. Sort key (optional) orders items within the same partition key, enabling range queries (e.g., all orders for a customer, sorted by date). Together they form a composite primary key.

Q: On-demand vs Provisioned capacity?
On-Demand: pay per request, auto-scales instantly, good for unpredictable/spiky traffic — I used this since my Lambda-triggered workload was bursty and unpredictable.
Provisioned: you set RCU/WCU capacity ahead of time (cheaper at steady, predictable high volume), with auto-scaling to adjust within limits, but can throttle if traffic spikes faster than auto-scaling reacts.

Q: What's a hot partition and how do you fix it?
It's when one partition key gets disproportionate traffic (e.g., a single popular request ID or a date-based key where "today" gets all the writes), causing throttling on that partition even though overall table capacity is fine. Fix: redesign the key to spread load — e.g., add a random suffix/shard number to the partition key ("write sharding"), or choose a higher-cardinality key.

Q: How would you support "all requests by a user in the last 7 days"?
Add a **Global Secondary Index (GSI)** with userId as partition key and timestamp as sort key. The base table stays optimized for lookup-by-request-ID, while the GSI supports the

"recent by user" access pattern — this is the core DynamoDB design principle: model a table per access pattern, not per entity.

Q: DynamoDB Streams — how and why?
Streams capture an ordered, time-sequenced log of item-level changes (insert/update/delete) for up to 24 hours, which can trigger a Lambda automatically. I'd use this to, say, trigger a notification or a downstream aggregation whenever a new result is written. Ordering is guaranteed **per partition key**, not globally across the table.

Q: What is optimistic locking / conditional writes in DynamoDB?
Using ConditionExpression on a write (e.g., attribute_not_exists(id) or checking a version number matches before updating) to prevent race conditions — if two processes try to write the same item concurrently, only one succeeds and the other gets a ConditionalCheckFailedException, which you handle by retrying with fresh data.

---

5. AMAZON BEDROCK

Q: What is Bedrock and why use it instead of calling a model API directly?
Bedrock is a fully managed service to access foundation models (Claude, Titan/Nova, Llama, Mistral, etc.) via a single unified API, without managing infrastructure or separate vendor relationships. Benefits over calling providers directly: everything stays inside your AWS account/VPC (better for compliance/security), unified IAM-based access control, no data used for training by default, and you can swap models without changing your integration code much.

Q: How do you invoke Bedrock from Lambda?
Using the AWS SDK's bedrock-runtime client, calling InvokeModel (or InvokeModelWithResponseStream for streaming) with the model ID and a JSON payload matching that model's expected input schema, then parsing the JSON response.

Q: How do you handle Bedrock throttling under high concurrent Lambda invocations?
Implement exponential backoff retries on ThrottlingException, and architecturally, decouple the calls using SQS or Step Functions with a concurrency cap (Map state's MaxConcurrency) so you don't fire more concurrent requests than Bedrock's account-level TPS/TPM quota allows. For predictable heavy load, you can also provision **Provisioned Throughput** for guaranteed capacity.

Q: How do you keep costs under control with Bedrock?
Pick the smallest model that meets quality needs for each task (e.g., a lightweight model for classification, a larger one only for complex generation), cache repeated/identical

prompts, cap max output tokens, and batch where possible. Token-based on-demand pricing means both input and output length directly drive cost, so prompt design matters.

Q: What are Bedrock Guardrails and Agents?
Guardrails let you define content filters (e.g., block harmful content, PII, or off-topic responses) applied to both prompts and model outputs, independent of which model you use. Agents let a model autonomously plan and call other APIs/tools/functions to complete multi-step tasks (like a mini orchestrator), versus a plain InvokeModel call, which is a single request/response with no autonomy.

Q: How do you prevent a silent model behavior change in production?
Always pin the exact model version ID in your Bedrock calls (not just a generic model family alias) so an AWS-side model update doesn't silently change your app's outputs.

---

6. AMAZON EC2

Q: On-Demand vs Reserved vs Spot vs Savings Plans?
On-Demand: pay per second/hour, no commitment, most expensive per unit — good for unpredictable or short-term workloads. Reserved: 1–3 year commitment for a specific instance type/region, up to ~72% discount, good for steady baseline load. Spot: bid on unused EC2 capacity, up to 90% discount, but AWS can reclaim it with a 2-minute warning — good for fault-tolerant/batch workloads. Savings Plans: similar discount to Reserved but flexible across instance families/regions, committing to a $/hour spend rather than a specific instance.

Q: Security Group vs NACL?
Security Groups are stateful (return traffic automatically allowed) and operate at the instance/ENI level, only support "allow" rules. NACLs are stateless (must explicitly allow both inbound and outbound) and operate at the subnet level, support both "allow" and "deny" rules — useful for explicitly blocking a malicious IP range at the subnet boundary.

Q: Why would you need EC2 in a mostly-serverless project?
Cases like: hosting a self-managed component with no serverless equivalent (e.g., a vector database not available as a managed service), long-running processes exceeding Lambda's 15-minute limit, workloads needing GPU access for custom model hosting, or needing full OS-level control for compliance/legacy dependencies.

Q: How do you achieve high availability with EC2?
Put instances in an Auto Scaling Group spanning multiple Availability Zones, behind an Application Load Balancer that health-checks each instance and routes traffic only to

healthy ones. ASG replaces unhealthy instances automatically and can scale in/out based on CloudWatch metrics (CPU, request count, custom metrics).

Q: Spot instance gets reclaimed mid-job — how do you handle it?
AWS sends a 2-minute interruption notice via the instance metadata service. Your application should poll for this and, on receiving it, checkpoint current progress (e.g., save state to S3/DynamoDB) so the job can resume on a new instance rather than losing all progress.

---

7. INTEGRATION QUESTIONS (highest BR focus)

Q: Walk me through your whole architecture end to end.
(Answer this with your actual flow — roughly:) A file lands in S3 → event notification triggers Lambda → Lambda either directly calls Bedrock or hands off to a Step Functions state machine for a multi-step workflow (extract → call Bedrock → post-process) → results are written to DynamoDB with idempotent conditional writes → downstream consumers (another Lambda via DynamoDB Streams, or a UI polling/querying DynamoDB) pick up the result. I chose this because it's fully event-driven, scales automatically with load, and I only pay for actual usage rather than idle infrastructure.

Q: Why DynamoDB and not RDS here?
My access patterns were simple, high-throughput key-based lookups (by request/object ID, and by user for recent history via a GSI) with no need for joins or complex transactions — DynamoDB gives predictable low-latency performance at any scale without me managing scaling, whereas RDS would need more operational overhead to scale to the same throughput.

Q: What's your retry/idempotency story across the whole pipeline?
Every write to DynamoDB uses a conditional expression keyed on a unique ID so retried events (from S3 or Lambda's async retry) don't create duplicates. Step Functions retries with exponential backoff on known transient errors (like Bedrock throttling) and routes to a Catch/fallback state after max attempts, so failures are visible (written as a "failed" status) rather than silently dropped.

Q: How do you monitor this in production?
CloudWatch Logs and metrics for each Lambda (errors, duration, throttles), Step Functions execution history for tracing exactly which step failed, DynamoDB CloudWatch metrics for throttled requests/consumed capacity, and X-Ray for end-to-end tracing across Lambda → Step Functions → DynamoDB to see latency breakdown per hop.

Q: What's the first bottleneck at 100x scale?
Likely Bedrock's account-level throughput quota (tokens/min, requests/min) — I'd mitigate with Provisioned Throughput and by decoupling calls through SQS/Step Functions concurrency caps. Second candidate: DynamoDB hot partitions if the key design isn't spread well — mitigated by write-sharding or redesigning the partition key.

Q: Tell me about a real production issue you hit.
(Have your own real story ready here — this is the single most important question in a BR round. Structure it as: what broke → how you found it (which logs/metrics led you there) → what you changed → what you did to prevent recurrence.)